

Spam Classifier & Model Comparison Toolkit

Project Report

Course: Fundamentals of AI and ML

Date: September 17, 2026

Name: Kurra Sanjay

Registration Number: 25MIM10079

2. Introduction

Spam messages are unsolicited, often malicious text sent in bulk via SMS or email. They range from harmless advertising to phishing scams that attempt to steal personal or financial information. This project applies core Machine Learning concepts covered in the course — text feature extraction, supervised classification, model evaluation, and comparison — to build a working spam-detection toolkit that can be run entirely from the command line.

3. Problem Statement

Manually filtering spam does not scale with the volume of messages users receive daily. An automated, content-based classifier is required that can learn the linguistic patterns typical of spam messages (urgency, prize offers, suspicious links, requests for calls/replies) and separate them from legitimate (“ham”) messages with high accuracy.

4. Functional Requirements

- Load and validate a labelled dataset containing **label** and **message** columns.
- Clean the raw text and convert it into **TF-IDF numeric features**.
- Train three classical **ML models**:
 - Naive Bayes
 - Logistic Regression
 - Random Forest
- Evaluate each model on a **held-out test set** using:
 - Accuracy
 - Precision
 - Recall
 - F1-score
 - Confusion Matrix
- Print a **comparison table** and automatically identify the **best-performing model**.
- Persist the evaluation results as a **timestamped JSON report**.
- Classify a single **ad-hoc, user-supplied message** from the command line

5. Non-Functional Requirements

Performance	TF-IDF feature extraction is capped at 3000 features (config.MAX_FEATURES) to
Reliability	Custom exceptions (DatasetError, ModelNotFoundError, ModelNotTrainedError) cat
Maintainability	Each concern (config, logging, preprocessing, modeling, evaluation, CLI) lives in its
Logging / Monitoring	A shared logger writes timestamped INFO/ERROR messages to both the console and
Scalability	New models can be added by registering them in model.py's _MODEL_REGISTRY
Usability	A single, documented CLI command (python -m src.main) runs the entire pipeline; --
Security	Input validation in preprocessing.py rejects malformed rows/columns before they rea

6. System Architecture

The system follows a simple layered architecture: a CLI layer accepts user commands, delegates to the preprocessing module to prepare data, passes the features to the model-training module, and finally to the evaluation module for scoring and reporting. Config, logging, and exception handling are cross-cutting concerns used by every layer.

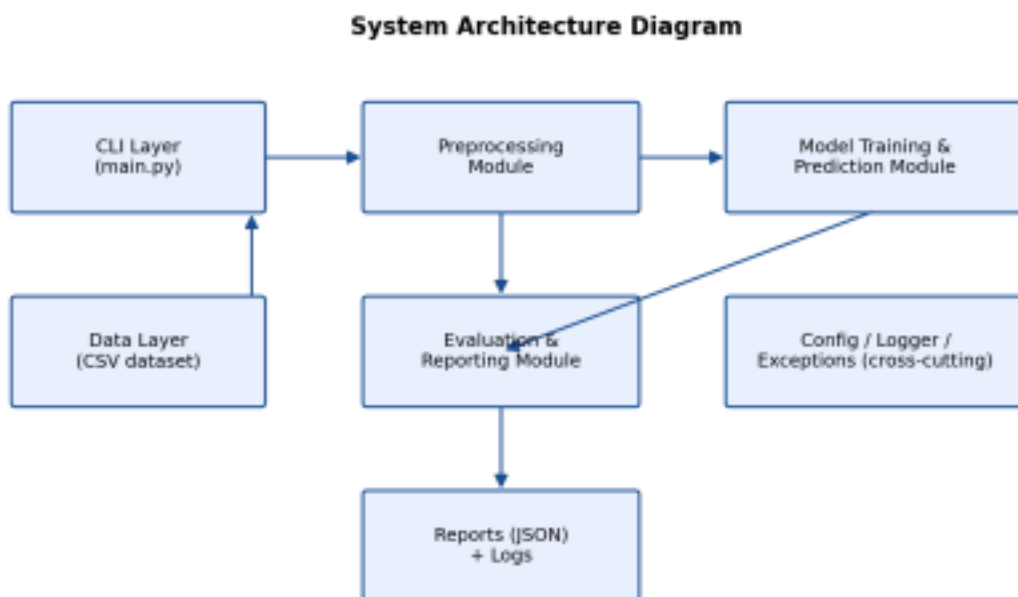


Figure 1: System Architecture Diagram

7. Design Diagrams

7.1 Use Case Diagram

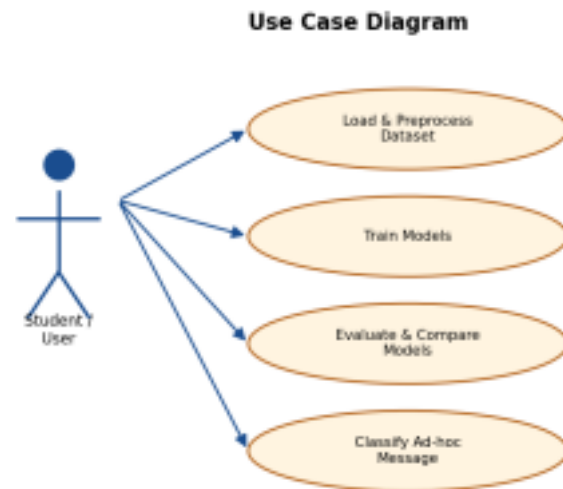


Figure 2: Use Case Diagram

7.2 Workflow Diagram

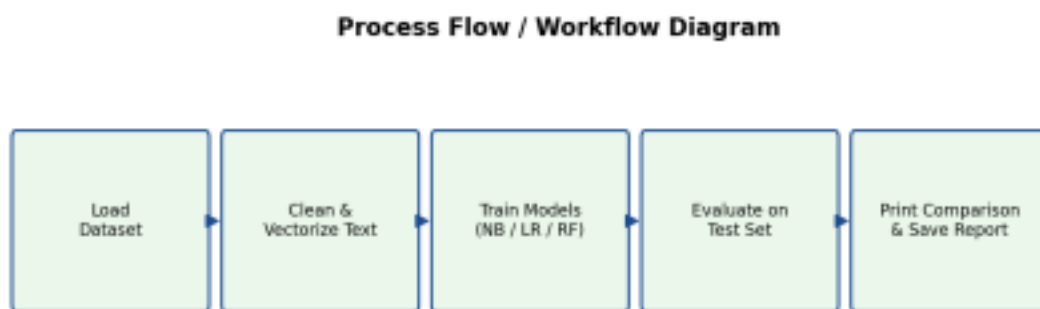


Figure 3: Process Flow / Workflow Diagram

7.3 Sequence Diagram

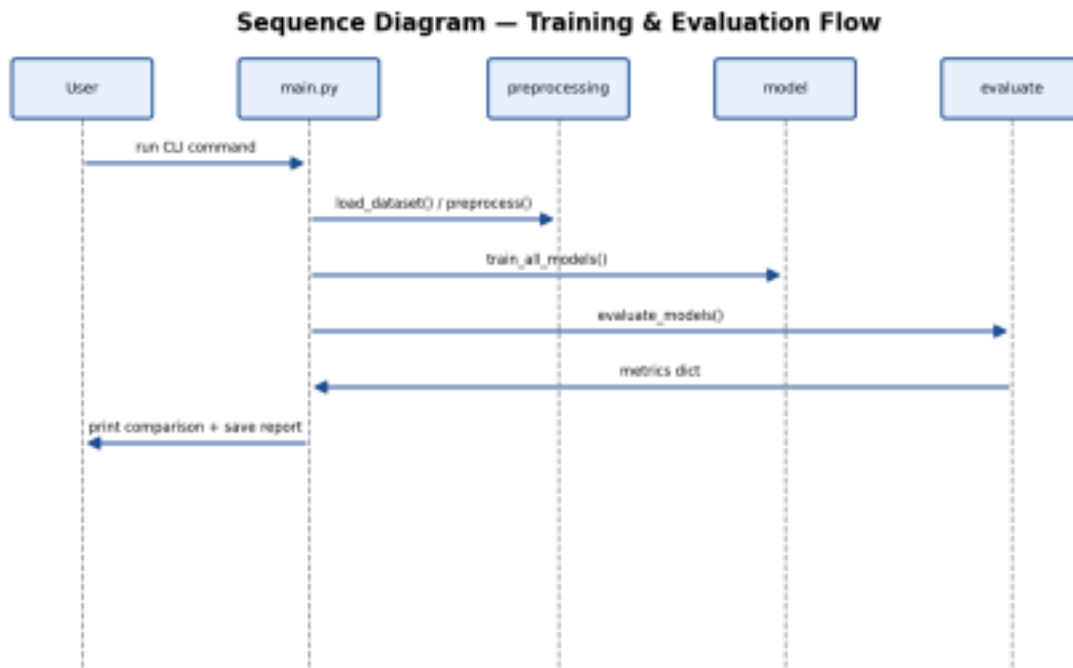


Figure 4: Sequence Diagram — Training & Evaluation Flow

7.4 Class / Component Diagram

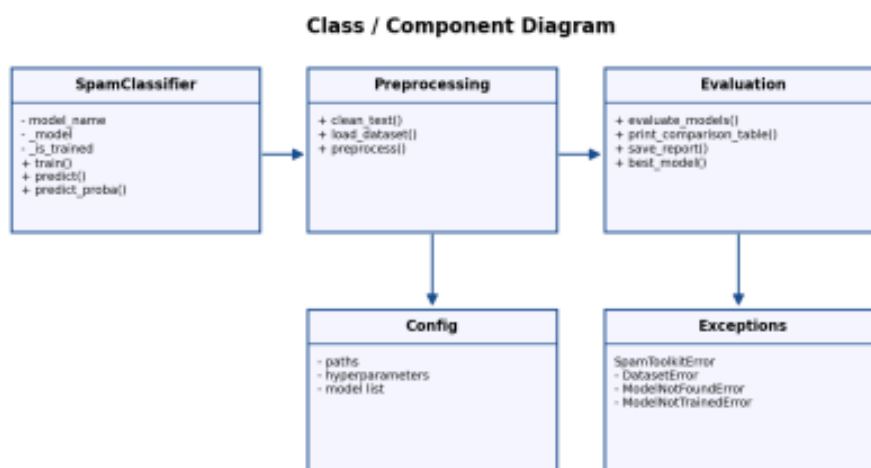


Figure 5:

Class / Component Diagram

No ER/database schema diagram is included because this project persists data to flat CSV/JSON

files rather than a relational database; this is noted as a possible future enhancement in Section 14.

8. Design Decisions & Rationale

- TF-IDF over raw word counts: TF-IDF down-weights very common words, giving more informative features for short, informal messages.
- Three complementary models: Naive Bayes (fast, strong baseline for text), Logistic Regression (linear, interpretable), Random Forest (non-linear, robust to noise) — comparing them demonstrates model-selection skills rather than committing to a single algorithm.
- CLI over GUI: satisfies the assignment's command-line executability requirement and keeps the project dependency-light and easy to evaluate/automate.
- Custom exception hierarchy: makes failure modes explicit (bad dataset vs. untrained model vs. unknown model name) instead of relying on generic tracebacks.
- Central config.py: avoids hard-coded constants scattered across files, so hyperparameters can be tuned in one place.

9. Implementation Details

The project is implemented in Python 3 using pandas and scikit-learn. **preprocessing.py** loads and validates the CSV

dataset, lowercases and strips punctuation/digits from each message, and fits a TfidfVectorizer (max 3000 features) before splitting into stratified train/test sets (75/25). **model.py** defines a SpamClassifier wrapper class and a registry dictionary mapping model names to scikit-learn estimators, so new algorithms can be added with one line. **evaluate.py** computes weighted precision/recall/F1 (appropriate even if classes become imbalanced), builds a confusion matrix per model, prints an aligned comparison table, and writes a timestamped JSON report to reports/. **main.py** wires everything together behind an argparse-based CLI, with try/except around the pipeline to surface SpamToolkitError subclasses as clean, user-facing error messages.

Repository layout (5-10 meaningful modules, as required):

- **src/config.py** — Configuration constants
- **src/logger.py** — Centralized logging setup
- **src/exceptions.py** — Custom exception hierarchy
- **src/preprocessing.py** — Module 1: Data cleaning & feature extraction
- **src/model.py** — Module 2: Model training & prediction
- **src/evaluate.py** — Module 3: Evaluation & reporting
- **src/main.py** — CLI entry point
- **scripts/generate_sample_data.py** — Sample dataset generator
- **tests/test_toolkit.py** — Automated unit tests (8 tests)

10. Screenshots / Results

Console output from running **python -m src.main** on the

bundled sample dataset:

naive_bayes	1.0	1.0	1.0	1.0
logistic_regression	1.0	1.0	1.0	1.0
random_forest	1.0	1.0	1.0	1.0

All three models score very highly on the bundled sample dataset. This dataset is synthetically generated from a small set of templates (see Section 12), so scores this high are expected on this data and are not representative of performance on real-world messages — see Section 12 for discussion.

11. Testing Approach

Automated unit tests (pytest) validate each module in isolation: text cleaning behaviour, dataset validation (missing file, missing columns), the end-to-end preprocessing pipeline, and model training/prediction — including that the correct custom exceptions are raised for an unknown model name and for predicting before training. All 8 tests pass (**pytest tests/ -v**). Manual/system testing was also performed by running the CLI end-to-end with the bundled dataset and with custom ad-hoc messages to confirm sensible spam/ham predictions.

12. Challenges Faced

Sourcing a ready-to-use, offline labelled dataset was the main practical challenge, so a template-based synthetic dataset generator was written instead. This makes the project fully self-contained and reproducible, but also means the reported

accuracy is optimistic (the vocabulary of spam vs. ham templates is quite distinct). Choosing an evaluation metric that stays meaningful if classes become imbalanced (weighted precision/recall/F1 rather than plain accuracy) was another design consideration.

13. Learnings & Key Takeaways

- How to structure a small ML project into **clean, testable modules** rather than one script.
- Practical experience comparing **generative (Naive Bayes)**, **discriminative (Logistic Regression)**, and **ensemble (Random Forest)** classifiers on the same text-classification task.
- Why **TF-IDF** is a strong, lightweight baseline feature representation for short text.
- The importance of **validating input data** and raising specific exceptions early, rather than letting errors surface deep inside a pipeline.

14. Future Enhancements

- Replace the synthetic dataset with a **large real-world corpus** (e.g., UCI SMS Spam Collection) for realistic evaluation.
- Add a **persistence layer (SQLite)** to store classification history, with an accompanying **ER diagram**.

- Expose the toolkit through a **lightweight REST API or web UI** for non-CLI users.
- Add **cross-validation and hyperparameter tuning (GridSearchCV)** instead of fixed defaults.
- Experiment with **word embeddings or transformer-based models** as a stretch goal beyond the classical-ML syllabus.

15. References

- Pedregosa et al., “**Scikit-learn: Machine Learning in Python**,” JMLR 12, 2011.
- McKinney, W., “**Data Structures for Statistical Computing in Python**,” SciPy 2010 (pandas).
- **UCI Machine Learning Repository** — SMS Spam Collection Dataset (reference dataset for future enhancement).
- **Scikit-learn Documentation** — <https://scikit-learn.org/stable/>